Branched Multi-Turn GRPO for Code-Policy Generation on a Single GPU

Abstract

We train a small instruction-tuned LLM to emit Python control policies for sandboxed RL environments using a critic-free, on-policy multi-turn RL scheme. The model interacts with the environment over a two-turn chat-template protocol: a scratchpad or rehearsal turn, then a committed `policy` body whose scenario reward supplies the only scalar signal. We replace flat trajectory-level credit assignment with a $G_1 \times G_2$ branched rollout: G_1 first-turn variants per prompt, G_2 second-turn continuations per variant, scored at the leaves. Per-turn Monte Carlo value estimates yield per-turn advantages broadcast across the tokens of each turn, in the spirit of VinePPO but at chat-turn rather than intra-completion granularity. To fit the resulting trajectories into a 24 GB GPU we exploit the additivity of the token-summed clipped surrogate to decompose the loss across turns, bounding peak LM-head gradient memory to max(turn 1, turn 2) instead of their sum. The contribution is a clean, small-budget specialization of standard ingredients (GRPO, VinePPO, ArCHer-style hierarchical credit assignment) to the chat-template multi-turn code-policy setting, plus the per-turn loss decomposition that makes it tractable on a single L4.

1 Introduction

Reinforcement-learning fine-tuning of LLMs has converged on a small set of recipes: PPO with a learned critic, GRPO and its DAPO/CISPO variants which replace the critic with a group-relative Monte Carlo baseline, and process-supervision pipelines that train an auxiliary reward model. Most of this work targets single-turn reasoning, where the trajectory is a prompt followed by one completion and the reward is observed at the end. Extending the same recipes to multi-turn agentic settings exposes a credit-assignment problem: a single trajectory-level advantage broadcast to every model token cannot distinguish a strong first-turn strategy followed by a poor execution from the converse.

We study this problem in the concrete setting of *code-policy generation*: the LLM emits a Python `def policy(state)` body that is then executed against environment scenarios in a sandboxed subprocess. The artifact is the code; the LLM is removed from the inner control loop at deployment. We use two chat turns. The first either drafts a candidate `policy` (whose actual scenario reward is read back as a tool message) or runs a `main` scratchpad whose stdout is fed back; the second commits the scored `policy`.

Our training scheme has two ingredients. First, we replace trajectory-level credit assignment with a $G_1 \times G_2$ branched rollout and per-turn MC value estimates — the macro-step analog of VinePPO's intra-completion branching, with hierarchical baselines that vary by turn-1 prefix. Second, to make the resulting two-turn trajectories trainable on a 24 GB L4, we observe that the token-summed clipped surrogate is additive over disjoint token subsets sharing the same global normalizer, and exploit this to forward and backward turn-1 and turn-2 independently. Peak LM-head gradient memory drops from $O(B \cdot T_{\text{full}} \cdot |V|)$ to $O(B \cdot \max(T_1, T_2) \cdot |V|)$ at gradient cost zero; the parameter gradients summed across the two passes equal the single-pass gradient exactly.

The combination is a natural specialization of standard mechanisms rather than a new algorithm. We do not claim algorithmic novelty; the contribution is a clean assembly that fits on a single small GPU and the loss-decomposition trick that makes it possible.

2 Related work

Critic-free LLM RL. GRPO [1] and its token-summed variants (DAPO [2], CISPO) replace PPO's learned critic with a group-relative MC baseline. We use the token-summed form because its additivity over token subsets is what makes our per-turn loss decomposition exact.

Branching for credit assignment. The closest existing method is VinePPO [3], which estimates $\widehat{V}(s_t)$ at chosen intra-completion states via on-policy MC re-branching and uses it as a per-token baseline. We use the same mechanism but branch only at *turn boundaries* of a chat trajectory and broadcast the resulting advantage as a constant within each turn — group-relative within a turn, VinePPO-like across turns.

Hierarchical multi-turn agents. ArCHer [6] adopts the same macro-step decomposition (per-turn value, per-token policy update) but learns the per-turn value with a critic. We estimate it by MC over G_2 continuations, trading variance for the cost of fitting a value head. ReBN [5] replaces GAE with batch-normalized MC returns; the hierarchical mean we use can be viewed as ReBN partitioned by turn-1 prefix. RAGEN's StarPO [4] is the canonical flat-baseline critic-free MC scheme for multi-turn LLM RL; setting $G_2 = 1$ collapses our scheme to it.

$G_1 \times G_2$ tree rollouts. The same tree shape appears in process-reward-model training pipelines such as Math-Shepherd [7] and OmegaPRM [8], where the per-prefix MC mean labels intermediate steps for a learned reward model. We use the tree as a policy-gradient baseline rather than as a labelling oracle.

Code policies. Voyager [9] and FunSearch [10] also produce executable code as the artifact, but rely on frozen-LLM prompting plus skill-library accumulation or evolutionary mutation respectively. We adapt the LLM by gradient-based RL of LoRA weights and add a scratchpad turn between prompt and committed code.

3 Method

3.1 Setup

The agent is a language model π_θ over chat-template trajectories. The base is `Qwen2.5-Coder-3B-Instruct` with frozen weights W; θ parameterizes a LoRA adapter $W' = W + AB^\top$. A task $\mathcal{T}$ specifies environment source code, S scenarios $\{\sigma_i\}$ that vary the seed and hidden parameters, and a per-scenario reward. The scoring oracle runs a candidate policy body in a sandboxed subprocess against each scenario and returns $R(\text{code}; \mathcal{T}) = \sum_i r_i(\text{code}; \sigma_i)$. Validation failures, errors, and timeouts on σ_i score the *error floor* $r_i = -200$, calibrated below any running-but-failing trajectory — otherwise errored rollouts ($r = 0$) outrank running-but-poor rollouts and the gradient rewards crashing.

3.2 Two-turn protocol ($N = 2$)

Each turn the model emits one fenced code block tagged `policy` or `main`. `policy` defines `def policy(state)`, which the tool executes across all scenarios and reports the summed reward as a user-role message. `main` is arbitrary scratchpad code with no environment access; the tool returns truncated stdout. The final turn must be `policy`; `main` or untagged final turns are

floored. Malformed intermediate turns are likewise floored and surfaced as a textual rejection. The full trajectory is $\tau = (s,\, y^{(1)},\, m,\, y^{(2)})$, with $y^{(1)}, y^{(2)}$ the two completions, m the user-role tool message after $y^{(1)}$, and $R(\tau) = R(\text{code in } y^{(2)})$ the only scalar reward.

3.3 Token-summed clipped surrogate

For trajectory b at position t, with importance ratio $\rho_t^b = \pi_\theta / q$ and per-token advantage A_t^b,

$$\mathcal{L}(\theta) = -\frac{1}{N_{\text{tok}}} \sum_{b,t} \mathbf{1}\{\text{model token}\} \cdot \min\big(\rho_t^b A_t^b,\, \text{clip}(\rho_t^b,\, 1-\epsilon_{\text{lo}},\, 1+\epsilon_{\text{hi}}) A_t^b\big), \tag{1}$$

with N_{tok} the count of model tokens in the global batch.[1] (1) is *additive over disjoint token subsets*: for any partition of model-token positions, the loss equals the sum of partial losses computed with the same N_{tok}. This is the property the per-turn decomposition (§3.6) exploits.

3.4 Branched rollout

For each prompt s, sample G_1 turn-1 variants in one `model.generate` call: $y_i^{(1)} \sim q(\cdot \mid s)$, $i = 1, \ldots, G_1$. For each $y_i^{(1)}$ execute the tool to obtain m_i, then sample G_2 continuations $y_{i,j}^{(2)} \sim q(\cdot \mid s, y_i^{(1)}, m_i)$ in one call. Score each leaf to obtain $\{R_{i,j}\}$. Generate-call cost per prompt is $1 + G_1$, versus $2G_1G_2$ for an unbranched rollout with the same number of leaves.

3.5 Per-turn MC value estimates and advantages

Two empirical values, both pure averages over the tree:

$$\widehat{V}_0 = \frac{1}{G_1 G_2} \sum_{i,j} R_{i,j}, \qquad \widehat{V}_1^{(i)} = \frac{1}{G_2} \sum_{j} R_{i,j}. \tag{2}$$

$\widehat{V}_0$ estimates the initial-state value; $\widehat{V}_1^{(i)}$ the value after committing to turn-1 variant $y_i^{(1)}$. Per-turn advantages, broadcast to every token of the corresponding turn:

$$A_i^{(1)} = \widehat{V}_1^{(i)} - \widehat{V}_0 \qquad \text{(over } y_i^{(1)}\text{)}, \tag{3}$$

$$A_{i,j}^{(2)} = R_{i,j} - \widehat{V}_1^{(i)} \qquad \text{(over } y_{i,j}^{(2)}\text{)}. \tag{4}$$

The turn-2 advantage uses the conditional mean given the turn-1 prefix rather than the global per-prompt mean: a poor turn-2 sample drawn from a strong turn-1 is recognized as poor instead of inheriting its prefix's above-average outcome. Tool-message tokens are masked via `tool_mask`; their advantage is zero.

3.6 Per-turn loss decomposition

A single forward over $[s \,|\, y^{(1)} \,|\, m \,|\, y^{(2)}]$ materializes an LM-head gradient of shape $(B,\, T_{\text{full}},\, |V|)$. With $|V| \approx 152{,}000$ this exceeds 24 GB at our trajectory lengths. Let $\mathcal{I}_1, \mathcal{I}_2$ index turn-1 and turn-2 model tokens. Since the tool mask zeroes the loss elsewhere and (1) sums over positions with a single normalizer,

$$\mathcal{L} = \mathcal{L}_{\mathcal{I}_1} + \mathcal{L}_{\mathcal{I}_2}, \qquad N_{\text{tok}} = |\mathcal{I}_1| + |\mathcal{I}_2| \text{ shared}. \tag{5}$$

We compute the two terms in separate forward/backward passes:

[1] This matches the DAPO and CISPO recipes; per-row averaging variants (original GRPO, BNPO, Dr.GRPO) divide by per-row token counts.

- **Pass 1.** Forward $[s \,|\, y^{(1)}]$ with `logits_to_keep`$= T_1^{\max}$, advantages $A_i^{(1)}$. Compute $\mathcal{L}_1$, backward, free.
- **Pass 2.** Forward $[s \,|\, y^{(1)} \,|\, m \,|\, y^{(2)}]$ but pack $[s \,|\, y^{(1)} \,|\, m]$ into the prompt slot and $y^{(2)}$ alone into the completion slot, so `logits_to_keep`$= T_2^{\max}$. Compute $\mathcal{L}_2$, backward, free.

By (5) the accumulated parameter gradients sum to the single-pass gradient exactly. Peak memory is max(pass 1, pass 2) since pass-1's graph is freed before pass-2's forward begins.

Restrictions. The decomposition requires the token-summed, single-normalizer form of (1); per-row averaging variants do not factor across $\mathcal{I}_1 \cup \mathcal{I}_2$. The implementation also needs $\beta = 0$ (a KL term to π_{ref} would require per-segment slicing) and `num_iterations`$= 1$ (so cached `old_per_token_logps` need no re-segmenting).

4 Experimental setup

Environment. A 2D puck-on-tilted-plane controller. The puck moves in $[-6, 6]^2$ under a constant hidden tilt (t_x, t_y) and linear drag μ; the policy receives $\{p_x, p_y, v_x, v_y, g_x, g_y\}$ and must drive the puck to a goal within 120 steps. Hidden parameters — tilt, drag, thrust limit $a_{\max}$, initial velocity — vary per scenario and are not in the state. The thrust envelope is set to $a_{\max} = \|\text{tilt}\|_2 \times 1.05 + \epsilon$ so a naive straight-at-goal policy fails because it cannot cancel tilt drift. Per-step reward is $-\Delta t \cdot \text{dist}(p, g)$ with a $+50$ bonus on reaching the goal; errored episodes score -200 per scenario.

Baseline gap. Two hand-written policies fix the headroom: naive aim ($\approx 26\%$ goal rate over 200 seeds) and PD-with-tilt-feedforward ($\approx 73\%$). The gap is the room LoRA fine-tuning has to exploit.

Model and compute. `Qwen2.5-Coder-3B-Instruct`, LoRA rank $r = 16$ on attention and MLP projections, on a single 24 GB L4 GPU. bf16 weights, gradient checkpointing on. We use $G_1 = G_2 = 4$ (16 leaves per prompt) and a per-turn token cap of 400.

Training. The token-summed clipped surrogate (1) with $\epsilon_{\text{lo}} = 0.2$, asymmetric upper clip, $\beta = 0$, and the per-turn advantages of (3)–(4). No reference KL, no critic, no GAE, no context truncation across turns.

Diagnostics. We log $\widehat{V}_0$ and the between-variant standard deviation of $\widehat{V}_1$, the within-variant standard deviation, and their ratio (the *signal ratio*); the per-turn advantage standard deviations; the fraction of trajectories at the error floor; and the count of `generate` calls per prompt (expected $1 + G_1$). The signal ratio guides allocation between G_1 and G_2 at fixed budget: > 1 means turn-1 variation dominates and a larger G_1 helps; < 1 means within-variant noise dominates and a larger G_2 helps.

5 Evaluating the per-turn credit assignment

The per-turn loss decomposition (§3.6) is purely a memory enabler — it does not change the gradient. The substantive claim that needs empirical support is the *credit-assignment* contribution: that hierarchical per-turn advantages (3)–(4) on a $G_1 \times G_2$ tree do better than a single trajectory-level advantage broadcast across both turns. We propose a six-arm ablation (Table 1), ordered from the weakest baseline to our method so each row adds exactly one ingredient over the previous.

Arm	Rollout structure	Advantage	What it isolates over the previous arm
A — Frozen, one-shot	N=1 inference, no training	—	Base model's prior on the code-policy interface
B — Frozen, multi-turn prompted	N=2 inference, no training	—	Does the two-turn protocol help at inference time? (Voyager-/FunSearch-style frozen-LLM use)
C — Single-turn GRPO	G flat N=1 rollouts, trained	Standard GRPO advantage	Does RL fine-tuning help at all?
D — Flat StarPO	G independent N=2 trajectories, trained	$A(\tau) = R(\tau) - \bar{R}$, broadcast to all tokens	Does multi-turn training help over single-turn? (RAGEN-style)
E — Branched, flat	$G_1 \times G_2$ tree, trained	$A_{i,j} = R_{i,j} - \bar{R}$ on all tokens	Does the tree alone help (more leaves per generate call), independent of credit?
F — Branched, hierarchical (ours)	$G_1 \times G_2$ tree, trained	$A_i^{(1)} = \widehat{V}_1^{(i)} - \widehat{V}_0$ on $y_i^{(1)}$; $A_{i,j}^{(2)} = R_{i,j} - \widehat{V}_1^{(i)}$ on $y_{i,j}^{(2)}$	The per-turn baseline contribution

Table 1: Ablation arms, ordered from weakest baseline (top) to ours (bottom). Trained arms (C–F) match leaf count $G = G_1 G_2 = 16$ so they see the same number of scored trajectories per step. D uses $2G$ `generate` calls per prompt; F uses $1 + G_1$. Report both equal-rollouts and equal-wall-clock comparisons. Frozen arms (A, B) use the same Qwen2.5-Coder-3B-Instruct base with no gradient updates, and serve as training-free references analogous to Voyager- or FunSearch-style frozen-LLM uses of the code-policy interface.

Primary metric. Held-out goal rate on a stress distribution (more extreme tilts/drag than train) at the best checkpoint, reported as median and IQR over ≥ 3 seeds per arm, plotted against gradient steps and against wall-clock. Statistical test: paired Wilcoxon over per-seed best-checkpoint goal rates; report effect size, not just p. The minimum convincing comparison is F > D at matched leaves.

Diagnostic checks. Two further measurements sharpen the mechanism story.

1. *Turn-1 quality curve.* Track the mean reward of turn-1 `policy` drafts (when turn-1 is in policy mode) as a function of training step. Under D the trajectory-level advantage projects onto turn-1 only through the global mean; under F, $A_i^{(1)}$ provides a direct gradient signal on turn-1 quality. F should show a steeper turn-1 reward rise than D.

2. *Signal-ratio conditioning.* The training loop already logs the per-prompt signal ratio $\widehat{\sigma}(\widehat{V}_1^{(i)})/\overline{\widehat{\sigma}(R_{i,\cdot})}$. Bucket prompts by their iteration-0 signal ratio. Ratio > 1 means turn-1 choice dominates the variance and per-turn credit should help; ratio < 1 means within-variant noise dominates and per-turn credit should not. F's advantage over D should grow with the bucket's signal ratio. If it does not, the mechanism is not doing what theory predicts and the headline claim weakens.

Failure modes to declare upfront. If F $\not>$ D under matched leaves, the hierarchical baseline does not pay off in this domain and the contribution reduces to "branching amortizes generate calls" (E vs D). If F $\approx$ E but both beat D, the gain is from the tree structure, not from the per-turn baseline. If the trained arms (C–F) do not clearly beat the frozen arms (A, B), RL fine-tuning is not buying anything over the base model's prior under our protocol, and the right framing for this work is prompt-engineering, not training. If B > A, the multi-turn protocol itself

helps even without training, and that gap should be reported as part of the headline. Either outcome is informative; we report whichever obtains.

6 Limitations

The scheme is restricted to $N = 2$ turns. Generalizing to $N > 2$ requires a recursive value bootstrap (each $\widehat{V}_n^{(\cdot)}$ becomes the conditional mean over the subtree below it), and the two-pass loss decomposition becomes an N-pass decomposition; neither is conceptually new but both add bookkeeping. We assume deterministic environments given the seed — under stochastic dynamics the leaf rewards become noisier estimates of state values and G_2 would have to grow accordingly. The $\beta = 0$, `num_iterations`$= 1$ restrictions are implementation choices that the same decomposition argument can lift with extra per-segment slicing of reference log-probabilities and cached sampling log-probabilities.

7 Conclusion

We described a critic-free, on-policy multi-turn RL scheme for code-policy generation that combines branched rollouts with per-turn MC value estimates and a per-turn loss decomposition. The algorithmic ingredients are individually standard; the assembly is small enough to train on a single 24 GB GPU, and the loss-decomposition trick is what makes the assembly tractable.

References

[1] Z. Shao et al. *DeepSeekMath*. arXiv:2402.03300, 2024.

[2] Q. Yu et al. *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. arXiv:2503.14476, 2025.

[3] A. Kazemnejad et al. *VinePPO: Unlocking RL Potential For LLM Reasoning Through Refined Credit Assignment*. arXiv:2410.01679, 2024.

[4] Z. Wang et al. *RAGEN: Training Agents by Reinforcing Reasoning*. 2025.

[5] Z. Liu et al. *GEM: Reinforcement Learning for Generalist Models*. 2025.

[6] Y. Zhou et al. *ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL*. ICML, 2024.

[7] P. Wang et al. *Math-Shepherd*. ACL, 2024.

[8] L. Luo et al. *Improve Mathematical Reasoning in Language Models by Automated Process Supervision*. arXiv:2406.06592, 2024.

[9] G. Wang et al. *Voyager*. TMLR, 2024.

[10] B. Romera-Paredes et al. *Mathematical discoveries from program search with large language models*. Nature, 2024.